

АНАЛИЗ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

1. Согласовать вариант задания с преподавателем, внимательно изучить индивидуальное задание к задаче (таблица 5.1) и пример.
2. Написать программный код реализации следующей задачи:
Реализовать две функции – итерационное решение задачи и рекурсивное.
Объяснить какой тип рекурсии использовался.
Посчитать максимальную глубину спуска, которая потребуется для нахождения ответа (N – целое число).
3. Ввод данных пользователем реализовать с клавиатуры, а вывод результатов выполнения программы – в консоль (на экран).
4. Проверить правильность вычислений на тестовых примерах, выполнив серию тестовых прогонов программы.

12	Вывести число Стирлинга второго рода ($s(n, k)$) : $s(0, 0) = 1$; $s(n, 0) = 0$; $s(0, k) = 0$; $s(n, k) = s(n - 1, k - 1) + k \cdot s(n - 1, k)$, для $0 < k < n$	Ввод: $n = 5, k$ $= 3$ Ответ: 25
----	---	---

ВЫПОЛНЕНИЕ ЗАДАНИЯ

Листинг программы:

```
#include <Windows.h>
#include <iostream>
#include <vector>
#include <string>
using namespace std;
HANDLE handle = GetStdHandle(STD_OUTPUT_HANDLE);

// Факториал
uint64_t fact(uint64_t n) {
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}

// Рекурсивное решение задачи.
// Прямая рекурсия
uint64_t recursionSolve(uint16_t n, uint16_t k) {
    if (n == k)
        return 1;
    if (n <= 0 || k <= 0 || n < k)
        return 0;
    return recursionSolve(n - 1, k - 1) + k * recursionSolve(n - 1, k);
}

// Итерационное решение задачи
uint64_t iterationSolve(uint16_t n, uint16_t k) {
    vector<vector<int>> dp(n + 1, vector<int>(k + 1, 0));
    dp[0][0] = 1;

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= k; ++j) {
            dp[i][j] = dp[i - 1][j - 1] + j * dp[i - 1][j];
        }
    }

    return dp[n][k];
}

// Проверка ввода на корректность
void validate_uint16_t(uint16_t& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName != "")
                cout << "\nВведите " << varName << ": ";
            cin >> sA;
            a = stoi(sA);

            break;
        }
        catch (invalid_argument ex) {
            SetConsoleTextAttribute(handle, FOREGROUND_RED);
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            SetConsoleTextAttribute(handle, FOREGROUND_RED);
        }
    }
}
```

```

        cout << "Введено слишком большое число" << endl;
    }
} while (true);
}

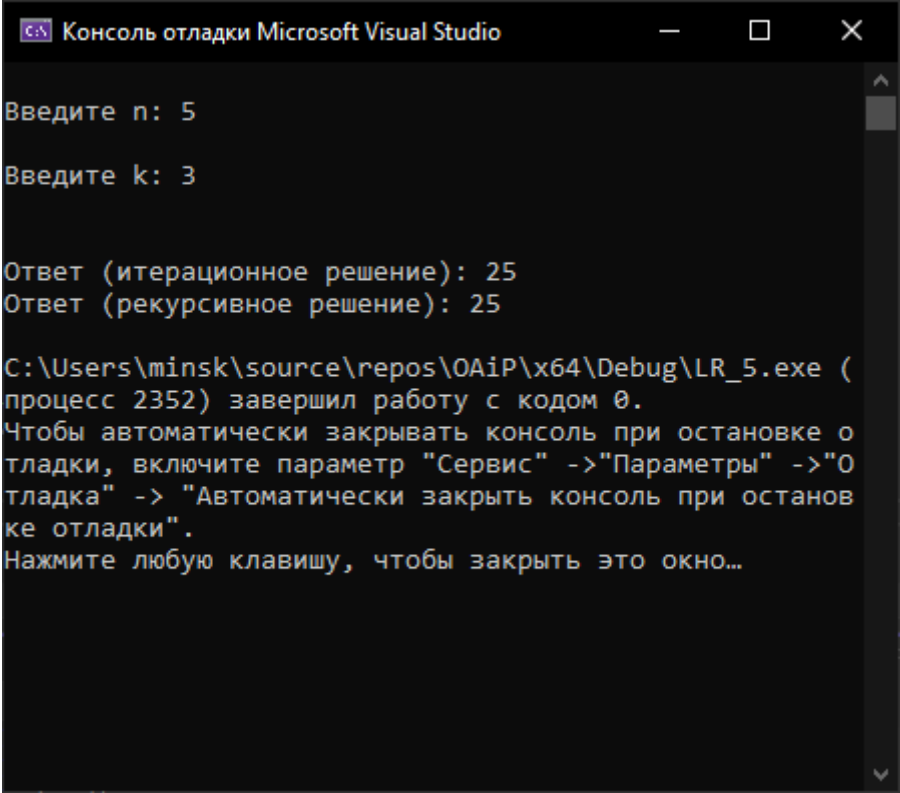
int main()
{
    system("chcp 1251");
    system("cls");
    uint16_t n, k;

    // Ввод s(n, k)
    validate_uint16_t(n, "n");
    validate_uint16_t(k, "k");
    system("color 07");
    cout << "\n\n";

    // Вызов подпрограммы решения и вывод ответа на экран
    cout << "Ответ (итерационное решение): " << iterationSolve(n, k) << endl;
    cout << "Ответ (рекурсивное решение): " << recursionSolve(n, k) << endl;
    return 0;
}

```

Результат выполнения программы (снимок экрана):



```

Консоль отладки Microsoft Visual Studio

Введите n: 5

Введите k: 3

Ответ (итерационное решение): 25
Ответ (рекурсивное решение): 25

C:\Users\minsk\source\repos\OAIp\x64\Debug\LR_5.exe (
процесс 2352) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке о
тладки, включите параметр "Сервис" ->"Параметры" ->"О
тладка" -> "Автоматически закрыть консоль при остано
вке отладки".
Нажмите любую клавишу, чтобы закрыть это окно...

```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Общие сведения о структуризации программы. Подпрограммы.

Структуризация программы в C++ обычно включает:

Разделение программы на функции: разделить программу на несколько функций, каждая из которых выполняет определенную задачу. Это позволяет легче навигироваться в коде, поскольку каждый участок кода отвечает за определенный набор задач.

Разделение программы на модули: разделить программу на несколько модулей, каждый из которых отвечает за определенную функциональность. Это позволяет легче обслуживать программу, поскольку каждый модуль может быть разработан, обновлен или замещен независимо от других модулей.

2. Определение пользовательских функций на C++.

В C++, пользовательские функции (user-defined functions) являются основными элементами структуры программы, которые помогают организовать код и обеспечивать его функциональность. В основном, пользовательские функции являются блоками кода, которые могут быть вызваны из любого места программы, где они определены.

Синтаксис определения пользовательской функции в C++ является следующим:

```
return_type function_name(parameter_list)
{
    // function body
}
```

В этом синтаксисе:

return_type - это тип данных, который функция возвращает.

function_name - это имя функции.

parameter_list - это список параметров, переданных функции.

function body - это тело функции, которое содержит инструкции, которые должны быть выполнены при каждом вызове функции.

3. Аргументы функции. Способы передачи. Типы и количество.

В C++, функции являются основными структурными элементами языка.

Аргументы функции являются данными, которые передаются функции, чтобы она могла выполнить свою задачу.

Способы передачи аргументов функции:

По значению: При передаче по значению, копия переданных данных создается, и функция работает с копией, а не с оригинальными данными.

По ссылке: При передаче по ссылке, функция получает адрес оригинальных данных, а не данные самих данных. Это позволяет функции изменить оригинальные данные.

Типы аргументов функции:

- `int`: целое число
- `double`: дробное число
- `char`: символ
- `string`: строка
- `const char*`: указатель на константный символ
- `const double*`: указатель на константное дробное число

Количество аргументов функции:

Функции могут иметь любое количество аргументов, зависящие от функции. В C++, функции могут иметь определенное количество параметров, которые передаются как один аргумент, например, в виде массива.

4. Вызов функции и механизм возвращения значений. Стек вызовов.

Вызов функции в C++ осуществляется с помощью ключевого слова `call` или `invoke`. В зависимости от того, как функция была объявлена, можно использовать одну из этих команд.

Пример объявления функции:

```
int add(int a, int b);
```

Пример использования функции:

```
int result = add(5, 7);
```

В C++ стек вызовов (`call stack`) является структурой данных, которая хранится в оперативной памяти компьютера. Он используется для хранения информации о текущих функциях, которые вызываются, а также для возврата к предыдущим функциям при завершении работы текущей функции.

Стек вызовов помогает программе управлять порядком вызовов функций, обеспечивая корректную оценку результатов и передачу данных между функциями.

5. Дайте определение понятию «рекурсия».

Рекурсия — определение, описание, изображение какого-либо объекта или процесса внутри самого этого объекта или процесса, то есть ситуация, когда объект является частью самого себя.

Рекурсивные функции - это функции, которые вызывают сами себя.

6. Примеры рекурсии: в природе, в технике, в социуме.

Рекурсия в природе: снежинка.

Рекурсия в технике: рекурсивные алгоритмы.

7. Дайте описание сути рекурсивного вычислительного процесса (определение рекурсивного алгоритма).

Рекурсивная функция – функция, вызывающая сама себя.

8. Достоинства и недостатки рекурсивного решения.

Преимущества: более простая запись некоторых алгоритмов

Недостатки: требования к размеру стека, скорость работы.

9. Представление алгоритма итерационным и рекурсивным способами. Итерационное решение предполагает использование прохода циклом. Рекурсивное решение предполагает использование рекурсии.

10. Проблема возврата из рекурсии. Контроль за глубиной рекурсии и переполнение стека.

Проблемой при отладке рекурсивного алгоритма является определение конечных условий, после которых рекурсивный алгоритм заканчивает работу.

11. Особенности использования локальных переменных при рекурсивном вызове подпрограммы.

Локальные переменные на каждом вызове рекурсивной функции создаются заново. Для того чтобы переносить переменную из итерации в итерацию, нужно делать ее параметром функции.

12. Виды рекурсии. Особенности их применения.

Хвостовая рекурсия. Это форма рекурсии, при которой рекурсивный вызов является последней операцией в функции перед ее возвратом.

Взаимная рекурсия. Это форма рекурсии, при которой две или более функции вызывают друг друга по очереди. Это можно использовать для алгоритмов поиска пути и им подобных.

Бесконечная рекурсия. Это форма рекурсии, при которой функция бесконечно вызывает сама себя, не достигая условия завершения.

Рекурсивные структуры данных. Это форма рекурсии, при которой структура данных, например, дерево или список, обращаются к себе самим.

Мемоизация. Это рекурсивный метод, при котором кешируются прошлые результаты и их возврат вместо повторного вычисления.

13. Анализ сложности рекурсивного решения. Рекуррентные соотношения.

Рекуррентное соотношение — это уравнение, согласно которому пятый член последовательности чисел равен некоторой комбинации предыдущих членов.

14. Использование рекурсии в решении прикладных задач. Примеры алгоритмов рекурсии в программировании.

Примеры рекурсивных алгоритмов:

- факториал
- число Стирлинга первого и второго рода
- числа Фибоначчи

15. Рекурсивные алгоритмы сортировки.

Сортировка слиянием, быстрая сортировка могут быть реализованы через рекурсию.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические сведения, реализовано программное средство, удовлетворяющее поставленной задаче, даны ответы на контрольные вопросы.